

Laboratório 01

Igualdade de valores e referências

Relatório de aprendizagem de Gabriel Falcão da Cruz · 12/08/2026

Status: implementação e execução concluídas. Falta apenas recuperar sem consulta a explicação do caso-limite `null == null` para concluir formalmente o laboratório.

Qual foi o objetivo?

Investigar, por meio de previsão e execução, o que o operador `==` compara em dois contextos diferentes:

- em tipos primitivos, compara os valores;
- em variáveis de referência, verifica se elas referenciam a mesma instância.

O laboratório também separou quatro operações que costumam ser confundidas: declarar uma variável, criar um objeto com `new`, atribuir uma referência e comparar referências.

O que foi praticado?

Primitivos iguais

`10 == 10` resulta em `true`.

Primitivos diferentes

`10 == 20` resulta em `false`.

Objetos distintos

Duas execuções de `new Pessoa()` criam duas instâncias; `==` resulta em `false`.

Referência compartilhada

`terceiraPessoa = primeiraPessoa` copia a referência; `==` resulta em `true`.

```
Pessoa a = new Pessoa(); // primeira instância
Pessoa b = new Pessoa(); // segunda instância
Pessoa c = a;             // copia a referência de a

a == b // false: instâncias distintas
a == c // true: mesma instância
```

O que Gabriel demonstrou ter aprendido?

Gabriel previu corretamente os quatro resultados, implementou os experimentos e explicou que `c = a` não cria nem copia um objeto: copia a referência. Também identificou corretamente dois objetos e três variáveis de referência no exemplo Feynman.

A principal melhoria de linguagem é trocar “mesmo endereço físico no heap” por “mesma instância” ou “mesmo objeto”. A JVM administra internamente as referências, e Java não exige que elas sejam tratadas pelo programador como endereços expostos.

Quais benefícios este laboratório traz?

1. **Entrevistas:** permite responder com precisão à pergunta clássica sobre `==` versus igualdade de objetos.
2. **Menos bugs:** evita usar `==` esperando comparar o conteúdo lógico de objetos.
3. **Base para coleções:** prepara o entendimento de `equals()`, `hashCode()`, `HashSet` e `HashMap`.
4. **Leitura de código:** ajuda a reconhecer quando uma atribuição cria um objeto e quando apenas compartilha uma referência.
5. **Depuração:** facilita compreender alterações observadas por duas variáveis que referenciam a mesma instância.

Último fechamento: caso `null`

```
Pessoa a = null;
Pessoa b = null;
System.out.println(a == b); // true
```

O programa imprime o resultado booleano da comparação, não o conteúdo isolado de uma variável. Como ambas as referências possuem o valor `null`, a comparação com `==` resulta em `true`. Já chamar `a.equals(b)` causaria `NullPointerException`, pois `a` não referencia um objeto.

Próximo laboratório: String Pool

Exercício 02 — String Pool

Continuaremos no tópico Java Core 1.1. O objetivo será descobrir por que algumas comparações de `String` com `==` parecem funcionar, embora `==` continue comparando referências.

O experimento deverá comparar, sem antecipar a solução completa:

- dois literais com o mesmo texto;
- um literal e uma `String` criada com `new`;
- `==` e `equals()` nesses cenários;
- previsão, resultado observado e explicação do papel do String Pool.

Benefício: compreender um comportamento muito cobrado em entrevistas e evitar comparações frágeis de texto. O laboratório não será iniciado antes do fechamento do caso `null` do Exercício 01.